AI Code Review Platform

FastAPI Project

This project is a FastAPI application that provides a RESTful API with various endpoints for testing and demonstration p

SEVERITY	CONFIDENCE	HEALTH
98	100	72

Language	**Python**	Architecture	**Monolith**
Complexity	**medium**	Sec Posture	**poor**
Bugs Found	**10**	Vulns Found	**5**
Doc Grade	**C**	Prod Ready	**not ready**

Generated May 24, 2026 at 06:43 UTC

Powered by Groq Llama 3.3 · LangGraph · ChromaDB RAG

The FastAPI project codebase has a medium level of complexity and follows a standard monolith architecture. However, the code quality is fair, with 10 bugs and 5 vulnerabilities identified, including 1 critical and 2 high-severity issues. The project lacks comprehensive testing, particularly for error handling and edge cases, and has a poor overall security posture. Despite these findings, the project has a good foundation, with features such as automatic API documentation and support for multiple HTTP methods. To improve the codebase health, it is essential to address the identified bugs, vulnerabilities, and testing gaps.

Key Findings

Severity	Category	Finding
CRITICAL	Security	Missing Input Validation, Insecure API Key Storage, and Missing Rate Limiting vulnerabilities were identified, posing a
HIGH	Bugs	10 bugs were detected, including Uncaught ValueError, Potential null reference, and Logic error, which can cause the app
MEDIUM	Documentation	The project's documentation grade is C, with a README score of 70/100 and medium docstring coverage, indicating a need f
MEDIUM	Architecture	The project follows a standard FastAPI architecture, but could benefit from additional error handling and logging mechan

Project	FastAPI Project
Purpose	This project is a FastAPI application that provides a RESTful API with various endpoints for testing and
Architecture	Monolith
Languages	Python, YAML, TOML, Markdown
Frameworks	FastAPI, Uvicorn, Starlette, Pydantic
Complexity	medium
Entry Points	main.py
Notes	The project follows a standard FastAPI architecture, with a focus on simplicity and ease of use. The use

Codebase Strengths

✓ The project has a good foundation, with features such as automatic API documentation and support for multiple HTTP methods.

✓ The project is well-structured, following a standard FastAPI architecture.

Code Quality: **fair** · Testing Gaps: The code appears to be missing tests for error handling and edge cases, particul

Uncaught ValueError in broken_dep

HIGH

tests/test_dependency_after_yield_raise.py

Category: unhandled_exception

The broken_dep function yields a value and then raises a ValueError, but this exception is not caught or handled. This could lead to unexpected behavior or crashes.

```
yield 's' raise ValueError('Broken after yield')
```

Fix: Add a try-except block to catch and handle the ValueError

Potential null reference in catching_dep

MEDIUM

tests/test_dependency_after_yield_raise.py

Category: null_reference

The catching_dep function catches a CustomError, but if the error is not an instance of CustomError, it will be re-raised. If the error is not handled elsewhere, this could lead to a null reference.

```
except CustomError as err: raise HTTPException(status_code=418, detail='Session
error') from err
```

Fix: Add a broader exception handler to catch any unexpected errors

Logic error in test_broken_no_raise

HIGH

tests/test_dependency_after_yield_raise.py

Category: logic_error

The test_broken_no_raise function tests that the response status code is 200, but the broken_dep function raises a ValueError, which should result in a 500 status code.

```
assert response.status_code == 200
```

Fix: Update the test to expect a 500 status code

Type error in test_path_int_42_5

MEDIUM

tests/test_path.py

Category: type_error

The test_path_int_42_5 function tests that the path parameter is an integer, but the value '42.5' is a float, which will cause a type error.

```
response = client.get('/path/int/42.5')
```

Fix: Update the test to use an integer value

Potential race condition in slow_async_stream

HIGH

tests/test_sse.py

Category: race_condition

The slow_async_stream function uses asyncio.sleep, which can cause a race condition if multiple requests are made concurrently.

```
await asyncio.sleep(0.3)
```

Fix: Use a lock or other synchronization mechanism to prevent concurrent access

Uncaught exception in get_sync_context_b_bg

HIGH

tests/test_dependency_contextmanager.py

Category: unhandled_exception

The get_sync_context_b_bg function does not catch any exceptions that may be raised by the bg task.

```
tasks.add_task(bg, state)
```

Fix: Add a try-except block to catch and handle any exceptions

Potential null reference in context_b

MEDIUM

tests/test_dependency_contextmanager.py

Category: null_reference

The context_b function yields a dictionary, but if the dictionary is not properly initialized, it could result in a null reference.

```
yield state
```

Fix: Add a check to ensure the dictionary is properly initialized

Logic error in test_async_state

HIGH

tests/test_dependency_contextmanager.py

Category: logic_error

The test_async_state function tests that the state is updated correctly, but the test does not account for the asynchronous nature of the state update.

```
assert state['/async'] == 'asyncgen started'
```

Fix: Update the test to account for the asynchronous state update

Type error in test_allow_inf_nan_param_true

MEDIUM

tests/test_allow_inf_nan_in_enforcing.py

Category: type_error

The test_allow_inf_nan_param_true function tests that the allow_inf_nan parameter is True, but the test does not account for the case where the parameter is False.

```
response = client.post(f'/?x={value}')
```

Fix: Update the test to account for the case where allow_inf_nan is False

Potential anti-pattern in test_serialize_response_dataclass LOW

tests/test_serialize_response_dataclass.py

Category: anti_pattern

The test_serialize_response_dataclass function uses a dataclass to serialize the response, but this could be an anti-pattern if the dataclass is not properly defined.

```
return Item(name='foo', date=datetime(2021, 7, 26))
```

Fix: Review the dataclass definition to ensure it is properly defined

Critical	High	Medium	Low
1	2	1	1

Missing Input Validation

CRITICAL

tests/test_request_params/test_query/test_required_str.py

A03: Injection

The application does not validate user input, making it vulnerable to SQL injection and cross-site scripting (XSS) attacks.

```
read_required_str(p: str)
```

Fix: Use Pydantic models to validate user input and ensure that all input is sanitized and validated.

Insecure API Key Storage

HIGH

tests/test_security_api_key_cookie_optional.py

A02: Cryptographic Failures

The application stores API keys in plain text, making it vulnerable to unauthorized access.

```
APIKeyCookie: {"type": "apiKey", "name": "key", "in": "cookie"}
```

Fix: Store API keys securely using a secrets manager or environment variables.

Missing Rate Limiting

HIGH

tests/test_request_params/test_query/test_required_str.py

A04: Insecure Design

The application does not implement rate limiting, making it vulnerable to brute-force attacks.

```
read_required_str(p: str)
```

Fix: Implement rate limiting using a library such as slowapi or fastapi-rate-limit.

Insecure OAuth2 Configuration

MEDIUM

tests/test_security_oauth2_authorization_code_bearer_scopes_openapi.py

A07: Authentication Failures

The application uses an insecure OAuth2 configuration, making it vulnerable to authorization code interception attacks.

```
oauth2_scheme = OAuth2AuthorizationCodeBearer(...)
```

Fix: Use a secure OAuth2 configuration, such as using HTTPS and validating authorization codes.

Missing Logging

LOW

tests/test_tutorial/test_request_form_models/test_tutorial002.py

A09: Logging Failures

The application does not implement logging, making it difficult to detect and respond to security incidents.

```
response = client.post("/login/", data={"username": "Foo"})
```

Fix: Implement logging using a library such as loguru or structlog.

Overall Grade	**C**
README Score	**70/100**
Docstring Coverage	**medium**

Quick Wins

- Add docstrings to undocumented functions
- Create a contributing guide
- Add a changelog section
- Improve code comments for better understanding

README Improvements

Priority	Section	Suggestion
HIGH	Introduction	Add a brief overview of the project's purpose and features.
HIGH	Getting Started	Provide a step-by-step guide for setting up the project, including installation and configuration.
MEDIUM	API Reference	Create a dedicated section for documenting API endpoints, including descriptions, parameters, and response models.

#	Category	Effort	Impact	Action
1	Security	high	high	Address the critical security vulnerabilities, including input validation, API key storage, and rate limiting.
2	Bugs	medium	medium	Fix the identified bugs, including Uncaught ValueError, Potential null reference, and Logic error.
3	Documentation	low	medium	Improve the project's documentation, including adding docstrings to undocumented functions, creating a contributing guide, and adding a changelog section.
4	Bugs	medium	high	Enhance the project's testing, including adding tests for error handling and edge cases, and improving test coverage.
5	Architecture	medium	medium	Implement additional error handling and logging mechanisms to improve the project's robustness and maintainability.

Recommended Next Steps

1. Address the critical security vulnerabilities.
2. Fix the identified bugs.
3. Improve the project's documentation.
4. Enhance the project's testing.
5. Implement additional error handling and logging mechanisms.